TSP Solver — Trivial Method Progress Report

Authors: Amine Nakari, Mohammed Mehdi Berrahou
Course: Programming

1. Introduction

The goal of this project is to implement a program that solves a restricted version of the Travelling Salesman Problem: metric TSP on complete graphs. Solving this problem requires finding the shortest tour that visits every city exactly once and returns to the starting city. However, at this stage, we are only implementing the trivial method, which establishes an input-to-output system that lists cities in ascending order of their indices, without optimizing the tour length. The input can be either the lower triangle of a weight matrix, including its diagonal, or a list of coordinates. The Held–Karp algorithm will be implemented at a later stage to find an optimal tour.

This version was implemented by Amine Nakari and Mohammed Mehdi Berrahou. This submission contains the source code for the trivial solution along with this report.

2. Chronological Account of the Work

Date	Stage	Summary
September 21	Project setup	Set up the project and reused graph classes from the previous lab.
September 30	Matrix parser	Implemented lower-diagonal parsing and the trivial method (outputting to a file).
September 30	Coordinate parser	Implemented the coordinate parser and its trivial method (outputting to a file)
October 14	Cleanup	Cleaned up the code and adjusted file handling.
October 17	Integration	Combined both parsers into one program and organized the source files.
October 17	Testing and documentation	Added basic tests, corrected issues, and documented program usage.

2.1 Understanding the Requirements and Planning

We started by reading the project instructions and looking through the provided files to understand their different sections and how to run the program. The coordinate format seemed easier for this first stage: we only had to read the cities and their coordinates, without worrying about how distances were calculated. The lower-diagonal format was a bit more challenging because we had to rebuild the full matrix from its lower-diagonal representation.

We split the work as follows: Amine took care of the lower-diagonal matrix parser, and Mehdi took care of the coordinate parser.

2.2 Developing the Matrix Parser

When it came to developing the matrix parser, we first started by looking through the lower-diagonal instance files to understand how the weights were listed. We read the instance name, type, and number of cities before reaching the most important part: `EDGE_WEIGHT_SECTION`. The number of cities was used to create an empty matrix. We then rebuilt the whole matrix by reading one number for the first row, two for the second, and so on, then copying each value to its matching position in the upper half (since the matrix is symmetric). The `AdjacenceMatrixWeightedGraph` class from the previous lab was reused to store the completed matrix. The matrix was also saved in a `.matrix.txt` file for inspection; these exports were later removed. Initially, reconstructing the matrix and solving were done within the same file, but the solving and output code was later moved to `Trivial_Solver.cpp` when refactoring for cleaner code.

2.3 Developing the Coordinate Parser

We initially implemented the coordinate parser as a class called `parser`, with its declaration in `parser.hpp`, its methods in `parser.cpp`, and a separate `main.cpp` to run it. The class first read and stored the file's lines, then extracted the metadata and located `NODE_COORD_SECTION`. From there, it read each city's ID and x/y coordinates until reaching `EOF`. It also included a method to write the trivial solution, and the initial program used `berlin52.tsp` as its input. During integration, the class methods were combined into a function in `Coord_Parser.cpp`, keeping the same parsing steps. As with the matrix parser, the tour-writing code was later moved to `Trivial_Solver.cpp`. An extra condition was also added to handle the space before `EOF` in `ulysses16.tsp`.

2.4 Integrating and Organizing the Program

At first, the two parsers were two separate programs. However, when refactoring, the parsers were combined into one program that detected the input format from the section marker inside the file. We ended up with three main `.cpp` files: `Trivial_Solver.cpp`, `Coord_Parser.cpp`, and `Parser_Matrix.cpp`. Additionally, instead of writing from each of the parsers, both writing functions were moved to `Trivial_Solver.cpp`.

2.5 Testing and Corrections

At first, we tried the program on lower-diagonal and coordinate instances manually and checked that the resulting tour output was correct (ascending ID order, followed by `-1` and `EOF`). One issue that arose was that the `ulysses16.tsp` file had a space before `EOF`, so a condition was added so the file could be parsed correctly. We also added two automated parser tests for `gr17` and `berlin52`. This validation process was necessary to have correct output, as incorrect output can be generated if the parsing or output handling is wrong.

3. Code Structure and Design Choices

3.1 Organization of the Source Files

We organized the files according to what each part of the program does:

- `Parser_Matrix.cpp`: reads the lower-diagonal values and rebuilds the matrix, then returns the name and graph data.

- `Coord_Parser.cpp`: reads the information at the start of the file, followed by the city IDs and their coordinates.
- `Trivial_Solver.cpp`: contains `main()`. It reads the arguments, chooses the right parser, creates the trivial tour, and saves it to the output file.
- `Parser.hpp`: declares the two parser functions so the solver can call them. It also defines the structures used to pass their results back to the solver.
- Graph classes: these come from the previous lab. `Graph` describes the basic graph operations, `WeightedGraph` adds operations involving weights, and `AdjacenceMatrixWeightedGraph` stores our matrix.
- `tests/tests.cpp`: checks a few values read from `gr17` and `berlin52` to see whether both parsers work as expected.
- `CMakeLists.txt`: lists what needs to be compiled to build the program and its tests.
- `build-TSPsolver.sh`: lets us build everything with one command.
- `TSP-instances/`: keeps the input files together, separate from the source code.

We wanted to avoid keeping everything in one file. Each parser takes care of reading its own format, while the solver takes care of creating and writing the tour. Keeping these parts separate makes the code easier to follow and lets us work on one part without mixing it with the rest. Later, we can add new solving methods and keep using the same parsers. There is still only one executable to run, and it chooses the parser from the contents of the input file.

3.2 Data Representations

Matrix Instances: We used the `AdjacenceMatrixWeightedGraph` class to represent the full matrix. Each position `[i][j]` contains the distance between cities i and j. The matrix is rebuilt in memory, without creating another file. Once the matrix was reconstructed as explained previously, we used the `TSPData` structure to hold that graph, the number of cities, and the type of problem, with the parser returning the instance name as well.

Coordinate Instances: `CoordinateData` stores the metadata in a map and uses the three vectors `indexs`, `xs` and `ys`, with `indexs[i]`, `xs[i]`, and `ys[i]` holding the ID and coordinates of the city at position i. However, our coordinate parser does not calculate distances yet. The trivial method only requires the IDs of the cities.

We kept both representations because they already provide what the trivial method needs.

3.3 Interaction Between Components

The components interact in the following manner:

Matrix: `main()` in `Trivial_Solver.cpp` detects the format, and `parse_tsp()` returns an instance name and graph data, then `trivial_solution()` creates the tour, while `write_tour()` saves it.

Coordinates: As with the matrix, `main()` detects the format, `parse_coordinates()` returns the IDs and coordinates, `write_coordinate_tour()` then saves the IDs in the order they were read.

All these checks are done using the contents of the file rather than the folder names.

4. Trivial Method and Program Usage

4.1 Trivial Algorithm

The trivial algorithm visits cities in ascending ID order. With four cities, this gives 1 → 2 → 3 → 4 → 1. The return to city 1 means returning to the start. The tour is valid because every city is visited once before returning to the start, but it is not necessarily the shortest. Distances do not affect the order because the method only uses city IDs. For coordinate inputs, we assume cities are listed in ascending ID order.

4.2 Building and Running the Program

From the project root, we build the solver and tests with:

```
bash build-TSPsolver.sh
```

This creates `build/TSPsolver` and `build/tests`. To run the tests:

```
./build/tests
```

The solver takes the input file path and output file path as its two arguments. For example:

```
./build/TSPsolver TSP-instances/coord/berlin52.tsp berlin52.tour
./build/TSPsolver TSP-instances/lower_diag/gr17.tsp gr17.tour -m trivial
```

The trivial method is considered the default, so `-m trivial` is optional. No other solving method is implemented as of yet.

4.3 Solution File Format

The solution file is formatted as per the requirements:

The output starts with the tour name (`NAME`), `TYPE : TOUR`, and the number of cities (`DIMENSION`). After `TOUR_SECTION`, each city ID is written on its own line, followed by `-1` and `EOF`. The starting city is not repeated.

The file is saved exactly where the output argument specifies. The parent folder must already exist if the output path includes a folder.

5. Validation and Difficulties

We added two simple tests using values from the input files. For `gr17`, we check that the graph has 17 cities and that the distance between the first two cities is 633 in both directions. For `berlin52`, we check the number of IDs and coordinates, the first and last IDs, and the first city's position, (565, 575). These tests only read the files and check the data in memory; they do not create output files.

The tests check a few parser results, rather than every value or the full process of generating and writing a tour. The program currently supports only the trivial method, and coordinate distances are not

calculated yet because this method does not need them.

6. Reflections and Next Steps

In this project, we learned how to use classes to represent graph problems and how to parse input files and generate output files. At first, we built everything independently and went for different approaches (I used functions, while Mehdi used classes), but in the end we refactored everything while keeping the main parsing logic the same. The refactoring separated the tasks properly and made the code easier to debug. What remains is to implement the Held–Karp algorithm, another solving method, and the performance evaluation.

7. References and AI-Use Disclosure

7.1 References

We used the TSPLIB 95 documentation to understand the instance files and the tour format.

We also used C++ documentation throughout the project, especially for reading files and working with strings. Examples include `find()`, `substr()`, and `npos` for reading the headers, string streams for extracting values from a line, and `std::pair` for returning the instance name and data together.

7.2 Use of AI Tools

We used ChatGPT to help understand the problem and clarify parts of the requirements. We also used it sometimes when debugging, for example when dealing with index-out-of-range errors or looking for a suitable method to use. It helped explain C++ functions that made some tasks easier, such as checking whether a character was found using `std::string::npos`. ChatGPT/Codex also helped write the test file.

These contributions are marked in the code with the following comments:

In `Parser_Matrix.cpp`:

```
// ChatGPT assistance, 2026-09-30: check whether ':' was not found.
```

In `tests/tests.cpp`:

```
// Partial ChatGPT/Codex assistance, 2026-10-16...
```

AI (specifically ChatGPT) was used to suggest an outline for the report and proofread some paragraphs, mainly to correct grammar and improve sentence structure.

7.3 DALIA Self-Assessment

We developed the matrix and coordinate parsers and reused the graph classes from the previous lab to store the reconstructed matrix and implemented the trivial solution. Nevertheless, based on the activities described in Section 7.2, we assess that assistance as D2–D3: we used AI to support our understanding

and help with debugging purposes. Additionally, it aided in the testing and guided us on how to implement testing for certain functionalities.